

William Gebhardt: "An Introduction to NGC-Learn: A Computational Neuro

1:25:14

welcome William Gert to the fourth
active inference Symposium we're looking
forward to your session titled an
introduction to NGC learn a
computational neuroscience libr Library
over to you yeah so today I'm here to
talk about um The Knack the nor adaptive
Computing Labs uh python n Tool uh
called NGC learn uh that is really
focused on getting your ability to
prototype your ideas and work with your
ideas in the field of computational
Neuroscience quickly and efficiently in
Python um so before we're going to start
off with all this first we kind of want
to briefly cover what we mean when we
talk about computational Neuroscience so
this is just the study and investigation
of brain function using mathematical
computation and tools and theories um we
have a focus on uh biologically
plausible and biologically grounded
systems and neur uh neurons and neuronal
systems and we pay very high attention
to the uh constraints Dynamics uh that
are found in the real world when we talk
about these uh for instance we model
things like you know we have the
resistance for membrane potentials we
have currents we have a bunch of other
various uh internal components to
everything that make sure that we are
modeling Faithfully uh all of the
neurons and components that we can find
inside NGC learn um and it we don't just

take from humans uh in fact there's a couple of various things that we have that are based on uh findings uh for things like rat hippo campel neurons and whatnot for stuff like synaptic plasticity and transfers um and so yeah so we kind of look at this biological interpretation of the world and so one of the main things that we also like to look at is like what is realistic when we talk about an interpretation of our world and how biology looks at it and so like for instance our human eyes it's been studied that they process think information at about 60 frames a second we can certainly detect things that move through our visual field faster than that but as far as like visual updates and whatnot once you pass 60 frames a second or 60 updates per second we have a hard time distinguishing and then also processing for a lot of things in our brain is not just we see an image we process the image once and then we just sit around idle until a new image comes into our brain um a lot of the time what we'll end up with is we'll end up with a continuous stream of data coming into our brain and then we have to do continuous updates and processing on that and it's that continuous update and processing that we're really are trying to uh hone in on and identify here uh we also uh so we do all of this inside of a biological framing uh so we use neurons now these are not

necessarily neurons like deep neural networks like to talk about neurons where it's just an abstract point in space uh we actually model them as uh physical objects uh that have things like internal States um and that also uh embody what they look like in brains and just other neural structures in the body um we have our uh we have our like the cell the center of the cell where we store a lot of our values we have synapses that connect these to other neurons later down and one of the key things that our library very much highlights is the use of spikes in spiking neural networks in order to actually achieve all of this happening um what this means is that instead of working in continuous values where you have a real valued image coming into your network um and then it passes through your entire network as a bunch of other real valued numbers uh images as they enter into our networks tend to be converted into what are known as Spike trains uh which are just a bunch of discrete either ones or zeros on whether or not there is signal and then from uh from neurons to neurons in our systems we tend to transmit these things only as uh spikes so everything is dealt with with discrete signals rather than continuous signals um and then the other thing that we really heavily rely on in NGC learn is stateful modeling uh so as we're dealing with these models that are constantly taking in signals over time

um it's to be expected that we require many signals and many iterations over the signals uh before we will actually produce any given output what that means is that all of our neurons have internal States and are stateful and so we can for instance stop getting any input and we can actually still watch what happens to the neurons models as they basically return to their rest positions um and so we have our biological components that have our internal States and then these states do change over time and so we tend to think about all of the models that we build with NGC learn and just computational Neuroscience in general uh somewhat akin to State machines uh with the idea that we progress Through Time by changing the state over own model and then there's certain actions that happen that can affect our model and transform it from one state to another um and so that's kind of the basic framing behind what we're going for with this right we're not trying to be the next tensorflow we're not trying to be the next deep neural network library uh what we're trying to be is we're trying to be a library that is specifically catered to computational neuroscience and the biologically inspired biic systems um so why do we need this uh so build these complex systems has a lot of overhead if we need to make a system that tracks all of the different internal states of all models uh and can

communicate between different components easily uh that can run efficiently um so at the bottom here you can see here so in one of these models that we built early on uh we had a pretty standard d uh spiking neural network uh that was not big by neural network standards uh that took about about eight hours to do an Epoch of amnest um just and that is due to the fact that when we are looking at an individual image in amnest we have to go over it uh I think this model was go it was doing 200 forward passes effectively over it basically we take the image and we convert it into a spike Train That's 200 time steps long and feed it through a model um in addition to this um because in our brain we don't have really the concept of mini batching you don't perceive 18 Different World Views at a time um and then update off of them uh or you know 8,000 or 500 or however many big batch sizes have gotten to these days uh we do everything in a continuous online updating um and so because of that we can only look at one image at a time so if we're doing roughly 200 forward passes per image one image at a time this can get really really really slow um and up to eight hours for us to do an epac over mnist um and so when we looked at some of the other existing libraries out there like Brian into bindet snn torch and whatnot they all worked and they all have their strengths uh to be able to implement these systems but some of them very much operate uh like black

boxes with the idea that you can modify parameters on the outside um but then you're entrusting that your various existing Library uh is doing everything you expect it to mathematically inside um and what this also does is this makes it hard sometimes to actually either dig in and see what those Dynamics specifically are or even just want to modify some of those Dynamics and so what we wanted to focus on was building something that was incredibly modular uh but still fast right so you can usually you have this trade-off between you can have a system that's incredibly fast but incredibly Limited in what you can do with it or you can have a system that is incredibly modular and can do a bunch of things but GS to tr off speed and so we were kind of aiming for the middle of that with error more to while trying to expand it to make sure we didn't sacrific too much speed to be able to do all of the modular stuff that we wanted to be able to do um and so with that we have some highlights of NGC learn here so first off this is an open source project it's available on pip it's available on GitHub uh we are an active development base which means that if you write us messages on like if you put errors or mess notifications it GitHub we'll see them if you email us we will see them we will respond to them we can work with you on them um it is uh so NGC learn specifically uh is built using Jax uh which is basically built on the underlying Library called The

Accelerated linear algebra library or xlaa which is a bunch of C of cpython bindings uh wrapped around um basically C functions for doing linear algebra and what that allows and so those are really fast because we get to use c um and then on so Jax is built on one step on top of that which basically creates a bunch of numpy bindings for said cpython functions um and so JX has this really really uh cool and uh component to it uh called Justin Time compilation and what this allows you to do is you can put a pure method uh into Jax's J in time compilation and it will give you a pure method out but when you run that method it runs entirely in C it does not actually do anything in Python and so the advantage of that is if we can take our giant models State transition functions and make them be pure functions where you give it an initial State and it gives back to you the final State we can pass that into Jax's J in time compilation and basically take the entire model and its entire State transformation and we can do it really efficiently in C um and actually as a matter of fact we can take that and we can do state transitions uh hundreds of times before we ever return to Python and so this is kind of how we get that speed up and in our testing that same model that took eight hours to do an Epoch over mnist uh we got down to roughly 15 minutes uh 12 to 15 running on the same Hardware um for to do an Epoch over mnist which is you know 40

time speed up there uh there have been other examples that we've had that have gotten up to a 50 time speed up uh this is not me promising that you will achieve a 40 or 50 times speed up in your work this is just me saying that like sometimes you don't realize how fast it can be running um we also wrote this so that way if you want to be writing your own new components because we all know that you know P models are great we can work with those those are great but the fun part about science is actually getting to build new things and try out new ideas and new modules so we made it really easy to write new stuff uh write new components write new basic like write new learning rules write new you know neurons all of the stuff is really easy to write and they wire directly into the compilers without any problems um and then the one of the other things too that we have is that it interfaces with uh Intel's Loi 2's uh simulator uh now the Loi 2 is a neuromorphic chip uh developed by Intel for doing neuromorphic Computing uh this is basically taking spiking neural networks and putting them at purely a hardware level um which is where all of the Energy Efficiency that you've heard people talk about with spiking neural networks comes from uh is the fact that when you put it onto a neuromorphic chip it's really efficient um and having worked with physical LoIs and whatnot it can be somewhat of a struggle uh to

actually get those chips running and get them working well and so what that kind of comes down to is when you want and you actually get access to one of the physical pieces of Hardware you want to make sure that your model probably at least Works in their simulator that they provide um I will tell you that the simulator does not one to one match with reality but that is uh something that Intel knows and is aware of and I presume working on um but what we allow you to do is that we have a we have a sub project of NGC learn called NGC lava um and what that gives you is a couple changed object classes and a couple more restrictions that you have to follow while building things um and then after that you can run NGC models directly in the Lui 2 simulator um this means that you can take your model and you can run it on a GPU you can run it on a CPU and you can run it on the uh Lui simulator all without actually having to change any of your code base you're basically just like changing a flag somewhere on where it's running and that's all you have to do for that so how do we actually build models in NGC learn because now that I've pitched it to you is this great tool let's see if it holds up with how easy it is so overall it's a modular system right so the goal of NGC learn was to be incredibly plug and play and also it has to be effectively one giant State machine so we can see on the right here we have this context block now for us a

context is generally our model uh but sometimes you'll have multiple models that you define or some models that are defined inside of one giant overarching context it's just the container that holds everything now the parts that we're the most interested in the things that actually do all of the updates and hold all of the fun bits of research are the components um and so components tend to be consist up of a couple of things so we have parameters uh and we have compartments and then there are the cables that connect them and we'll talk more in detail about what each one of those are in a little bit but that so but the components are the new things right if you build a new neuron if you build a new weight Matrix with a new learning rule or whatever those are what you're building you're just building those little tiny parts and because of the way that they interface with one another is you don't have to worry about oh does this have the right number of outputs to line up with the right number of inputs and whatnot if you build the components in such a way that uh you follow just the standard practice that we've set out all of the data will flow from component to component without any problems so if we want to actually talk about the components these are the backbone of what you're doing so on the so on the left here I have the Leaky integrate and fire neuron now this is one of the most common neurons to see when you're starting out in spiking

neural networks uh it is the idea that you have uh an internal voltage that slowly builds up passes over a threshold and then depolarizes into msah Spike um and it does that by get taking in current and you know $V = IR$ and you do current times resistance and you get a voltage and it slowly builds up your voltage so inside of your LF neuron or our leaky integrating fire neuron we have a couple of fixed parameters now for us fixed parameters are values that don't change during runtime these are the static parts of your model but they're not but they're not fixed across all of your components and what I mean by that is that I can have a leaky Integra and fire neuron like with a voltage threshold of one and I can have another leaky Integra and fire on with a thrt threshold of two um and you can build both of those it works perfectly fine but the one and the two can't change um well it's running uh because just of the way that it all works out internally with the compiling but for instance here we have some fixed parameters we have neuronal units so when we talk about having a neuron or a component that is a neuron it is more in reality a layer of neurons so for us and neuronal units is just the number of leaky integrate and fire neurons that exist inside this little component or cluster of neurons uh they all have a voltage threshold um then we have to M RM a V reset and a VR test uh these are just all of the various parameters that

you need to get a leaky integrate by
neuron

running and then below all of these we
have the compartment values now this is
where you actually store the stateful
part of your component this these are
the values that change over time these
are the values that we are transmitting
to from component to component and so
for instance here in our leaky Integra
and fire neuron on the left we can see
that we have current coming in on the
left we have this voltage value that
kind of floats around in the middle
because we're not expecting to put to
have it as an output uh we have our
output Spike which is just whether or
not our voltage did Spike and then we
have time of last Spike uh which is just
well the last time step that it spikes
because there's multiple learning rules
that require that and so what this so
what we can see here is that if you can
kind of think about the various parts of
your model that you like to build the
neurons the weight matrices any other
auxiliary system that you have in your
model if you can kind of fit it to a
component it's incredibly
straightforward to get these to be built
and then once you have them in component
form you can then plug them into any
part of any
model so then uh what we're going to
talk about here is we're going to talk
about cables now cables can be a little
bit confusing at first uh so cables are
what actually connect our components

together so if we look at the top we have component a and we have component B and basically what this is is that and then they're connected with the cable and so we have our output compartment which is the brown box that's on the right side of component a and then we have our input compartment that's on the left side of the compartment's box for component B and then they're connected with this cable and so the idea is that data will flow from the output compartment of component a and get copied into the input compartment of component B and so this is basically how we get data to move from one to the other without necessarily having it know where it comes from so this was one of the main things that we wanted to make sure you uh was a founding principle of kind of what we were going for when we looked at the component and cables is that uh a component downstreams so case component B it doesn't necessarily need to know and it shouldn't know where the incoming values are coming from so these cables are all oneway cables you cannot go backwards up the cable uh basically the components are just told hey this is the value that has been transmitted along this line and thus this is the value that you have and you don't get to know where it comes from now what this does mean though is that sometimes sometimes what we'll have is inside we'll have a component that knows that it's getting values from two places for instance in the bottom explanation we

have component X and component y each one of them have an output but we're trying to put both of their outputs into the same compartment in component Z now if we were to do this without the summation that you can see what would happen is the value from component X would come down and it would get put into the compartment and component Z and then the output of component y would come down and it would also give put into the compartment of component Z effectively over writing component X and so but what we do have is we have this concept of operators which are just simple uh Transformations that can be done to the data that are being transmitted along cables so in this case summation uh this is you just have two you just have two pieces of data that all flow into this little summation operation that just flows into your output compartment um it's important to note that the value like the Intermediate values inside of things like summation are never stored anywhere they are purely just an inline operation that happens

yeah so that's all you actually need to build a model right is you have components and you wire them together that pretty much talks about everything you need to know for the physical parts of your model and how they talk to one another but that doesn't get us anywhere if I just hand VI a model that has a bunch of wires around it and a bunch of

values in it but I don't tell you how to actually change any of the values that's fairly useless um in addition to that it's not a particularly great you know just research project we all care about how do these models adapt over time we all care about that whole temporal and evolving nature of everything so with that in mind we want to talk now about how do we actually transition a model and uh what goes into that so for transitioning a models ACR so we have state transitions across just a component so if we just zoom in on what a component is we have component a here uh where we have our fixed parameters across the top here you can see we have four fixed parameters and then we have two different state transitions that we're going to allow we have the reset transition and then we have the advanced transition and so in this example here we can see that the reset transition takes two fixed parameters in and sets values via like the dash lines that we see sets values into all four compartments or we have the advanced transition that takes in again two fixed parameters but different fixed parameters than reset and then it also uses three of of the compartment values we can see the input compartment value on the far left and the two floating compartment values with their solid lines that go into advance and basically this is we have written an advance function so basically this is just we're

doing a state transition called advance that requires two fixed parameters and three dynamically changing compartment values Advance internally does some transformation of this data and it outputs two values one of those values gets looped back into the floating compartment that we see on the right and the other one gets sent to the output compartment that's also on the far right and that's really all these State transitions across components are when we write them internally and we'll look at some of what this looks like in a little bit um what we'll notice is that each one of these uh transitions is relatively simple right it's just you just take values in do TR do something to them and send values out and that is really again you'll see this throughout all of NGC learn that is basically how it all works under the hood is that we just we take values in we transform them we send values out and we don't necessarily need to know where those values are going or where the input values came from that we just know how to move them around and it's important to note here again that the parameter parts are fixed uh it's only the compartment values that we can change so you couldn't see Advance draw a dotted Arrow up to one of the fixed parameters because they are once again fixed so now that we know how to do a transition across one component what if we wanted to do a transition across two

components and I'm aware that the parameter little circles are missing their arrows the images got significantly too cluttered uh when I tried to draw those arrows in as the diagrams get smaller um but what we'll notice here is that we now can take a group of components and we can talk about taking the advance action across a group of components and so what this basically does is we will do the Advan operation or transition on along component a we will then transmit all of the data along all of the cables going between component a and component B the whatever that outputed value of advanced state was would get sent into its output compartment and then sent to component B and then component B would once again also do its Advanced transition in which it would take on all of its values and compute its own output value and you could chain these together at infinum there's no limit to how many of these you can chain together um I will say that eventually it's probably you know might want to look at making your model a little bit smaller but there's no actual limit to it uh there's also no limit to how complex you make these uh there's nothing that says that you can't have recurrent Connections in your uh cables uh there's also nothing that says that outputs or inputs need to actually have values you can clamp values to them um which is basically the process of we just set the value and then just let it go but there is one thing to note

especially when talking about recurrent things and recurrent connections between neurons and groups of neurons and uh what that is is that the order of operations in a state transition is deterministic and thus it's also something that each one only goes once so if you have component a wire into component B and then component B wire into component a it will still only execute a transition across a and then across B it or across B and then across a it will never do it will never actually Loop itself um it will always do one and then the other and so now all of this really really works itself down to compiling so the compiler of NGC learn um is not actually found in the main repo of NGC learn so NGC learn is built on top of what is known as NGC stimulation library or NGC Sim lib for short um and that is where all of the I don't want to say messy or ugly but that is where all of this code behind the scenes that are making everything work exists um and that is all of that Library can be found in that little tiny compiling arrow that we see down here now what we for the way compiling works though is we have our state transition right we have the state transition that we talked about in the previous slide and we want to turn it into a pure operation and what we mean by that is that it uh the same inputs will always produce the same outputs and no value outside of the outputs of the

method are ever
modified and so what how we can do that
is with the way that the transitions all
work is we can just pass in the entire
state of our model and then based on the
transitions inside each component they
know what compartments they have to be
looking at and so they will look at
those compartments and they will then
update and modify the values in the
model and then it'll move from that one
into the next one and from the next one
into the next one and so on and so forth
until it's actually done and so these
are these produce pure methods and if we
think back to what I talked about with
Jax all along at the beginning uh that
means that we can pass these compiled
methods into Jax's just in time
compilation and so this gives us is this
allows us to basically call Advance or
reset or any transition function that's
defined across all of components at what
like effectively at once uh in one
operation uh here we have it as Advance
the
model and so this is truly where the
speed up of NGC learn comes from um even
if you do not use uh Jax Jax or if you
don't use Jax's just in time compilation
you're debugging um it is still a fairly
large speed up just to go from python
being written like the way it is into
compiling into just running the compiled
operation um so if you really want to
look into and care about how are we
getting the speed that we're getting
look into the compilers um and once

again the compilers are actually located in NGC learn NGC Sim lib which still can be found through all of The Knack laabs GitHub Pages it is also public

so now what does this actually look like in code uh and so I wanted to take a brief look um because pretty pictures are really are you know a tool that helps us understand the thought process behind the models and they understand how the models actually work but sometimes people will show us really pretty pictures and then when you go to actually implement it in code it looks a whole lot different or things just don't quite or not quite as pretty of a picture uh so we do have a short section here where we're going to talk about all of the different code that we do have

so here's an example of a component in NGC learn uh specifically we are going to be looking at the DSE synapse now all a dense synapse is is it is a weight Matrix uh that has both a resistant scale value on it so just a constant multiplication scale factor being applied to everything um as well as a bias term that is added to everything now so here we have some compartment values on the left so as you can see uh defining compartments is incredibly easy so now uh everything right here where you see self dot is all inside of the Constructor of our object um so here we have self. inputs is just compartment and then some values for us that's the prev Val so that's whatever

we want that's a it's a matrix of all zeros of the correct shape and we have self. outputs which is just again another compartment defined with post vals and then we have our weights now weights are just our weight Matrix uh which is defined previously in the function um but it is just again simply compartment value and what this does a bunch of stuff under the hood but this basically actually Associates so when you build one of these objects when you build a DSE synapse and and construct it this will actually build a bunch of compart this will build three compartments under the hood and basically register them inside the D synapse and Link everything together correctly so that way we know that they exist and we can wire into them and it does a little bit more than just python knowing that the object has these fields in it they exist and they have their own set of properties um and then we have our fixed parameters which if look look exactly like just values that exist on a class right we have self. shape which and self. our scale which is just a resistance scaling and I'm sure everybody here who's worked with python is very familiar with what that looks like and so finally we have defining our state transitions now there's a couple things to note about this we'll notice that our state transition definitions are static methods and what they are really is they're just method these have to the pure methods to Define our state

transitions um so here we have advanced State and we want to still be able to use our scale inputs weights and biases inside of advanced State and have them all correlate to the values that exist inside of our component but due to the way that the compiler works is we still need this to all be a peer method so how do we combat that well we combine that by just doing effectively pattern matching where as long as your names for the various inputs so r scale inputs weights biases whatever as long as those values are found inside of the component that this transition is acting on uh it will automatically grab those values out of those components when it's doing the transition and so here we can see this is a pretty straightforward matrix multiplication with a scaling and a bias term

uh to compute outputs and we'll notice that once again outputs is something that is keyworded on the component and because when it's done resolving its transition it needs to know where to put the output value or in this case outputs of the state transition like what values are changed and so it knows how to put outputs back into self.

outputs and so this is kind of the key to how all of NGC learns compilation and just plug-and play models work with these State transitions is that you can Define these methods that can pull in values from the components that they are appi being applied to without actually having to have an instance of the

component itself now there's one other F
uh aspect of these that's not actually
shown in this example and that is
parameters at runtime so for instance
let's just say that we wanted to add the
current time to our output I don't know
why we would want to be doing that but
let's say we wanted to so if inside
Advanced State inside the parameter list
we had another value say time or T and T
doesn't exist on the component that this
transition is being applied to what
we'll see with this then is that late it
will and we'll see this a little bit
later in an example what it will want to
do is it will say hey I can't find all of
these values but this specific value T
is not found in this component you need
to give me T for whatever I need to do
for this state transition and so this is
how you can get runtime parameters to be
passed into your state Transitions and
modify them that
way so now that we have a component and
we kind of figured out how to define a
component and I should note that we can
Define as many of these State transition
as we want because they're not bound to
any specific
component we need to do something so how
do we actually Define these components
in a model because generally we want
multiple of them in a model a component
is one piece of a model is not very
useful so in order to do that we use
blocks in Python uh for us we call them
contexts uh so we do a with a context
and then we give our context a name in

this case we've named it model and so what we have here is this is a WID block and so what it what this does is this is just saying everywhere every time we Define a new NGC learn component or initialize a new NGC learn component inside of this width block it will automatically be added to the model so there's no model. register various built things no model. components list that you need to make sure is up to date nothing like that if we call heav and synapse here inside of this WID block the heav synapse exists on your model now what this does mean though is that the names of all of our synapses are required to be unique uh but I feel like that shouldn't be super challenging because in our models just for clarity and human readability we try not to name things the same for instance it wouldn't really make sense if I have three different parts of my model that I've named

Z1 so with that we can see that we can define components like this but what if we have complex models in which we have one overarching model with submodels that run maybe at different times or at a different frequency and we want to be able to have those still be a part of this bigger model the bigger scope of the model but run on their own well luckily for you we can do that so components are sorry contexts are fully nestable you can Nest them as deep as you want and when you actually look at the structure internally you'll notice

that it follows very similar to a file structure where everything just ends up with paths based on context name and so you can Nest these as deep as you want generally in my own personal use I've never found really a need to go more than an extra layer deep but there's nothing that stops you from going three four five layers deep um and so as we can see here though it's incredibly straightforward to actually build the model uh this here has built a model with two heavy and synapses in it now I'll give you a brief note there's a bunch of other components that are defined below this to get the model to do something um but those all look pretty much exactly the same as this just with different objects and different components being built at every

line so now we have our components we have our components defined inside of our context but they don't do anything they don't they don't interact with one another if we think back to the way that we get components to interact with another we have to wire them between each other right we have to get data to flow from a heavan synapse into an LF cell we have to get it to flow from an input placon neuron into a weight Matrix into an LF cell we have to get all of this data to flow so we do that with cables and luckily defining cables is incredibly straightforward so we have overridden the leftand bitshift operator basically uh to say hey

if we're going to read the top here so unfortunately we end up reading right to left with this is we're going to take the ZZ outputs whatever is in the outputs of z0 and we want to wire it directly into W1 which is one of our weight matrices inputs or inp and we just write that line and that cable has been connected everything under the hood is

done and then and I should not I should mention all of this is still inside the WID blocks that builds the Context of our model anytime we're dealing with building parts of our model it is all done inside of the wi

block uh and then on the next line we can take z1i spikes and we can output we can send that into W1 I's inputs and we'll notice that this pattern of just taking an output and wiring it into an input um is pretty easy to follow and so if you build your component with the inputs that you're expecting and the outputs You're Expecting and you make it and you make it really clear to yourself what you need to do with that when it comes to wiring up your model it should be incredibly

straightforward but if we think also back to the wiring example there was a point where I put two outputs and wired them into one input so we can see that actually happening inside the second part of this where we connect two compartments into a single destination so we can do this with for us the operation summation and so what this

will do is this will add together any number of compartments that we put here so in here we just have it as two but if we had four compartments that we wanted to wire into there uh it would be easy now one of the advantages of doing this instead of just having two input compartments inside of Z1 is this allows for a variable number of connections so if inside of one model that you define you need Z1 to have three in or three values being added together or inside of a different model that you're defining you have Z1 needing to have seven inputs going into it you don't have to build new components there's no additional logic that you have to do you just add them to this list of parameters that are being passed in and you're done and another thing to note is that these parameter chains are entirely nestable as well so for instance if we wanted to invert or negate uh for W1 I's output value so it would effectively be W1 out minus W1 iie outputs you just put negate around it and you can just Nest them down so if you wanted to sum a bunch of things together and then negate it you can if you want to negate two of them and add them it all works um and I should also again mention it does not actually store these values anywhere so they are never actually inspectable outside of just looking at the value of the compartment so compiling so compiling an NGC learn we didn't want it to be a painful

process we didn't want it to be something that you had to make sure that everything was like touched the specific right way where you're holding it in with all your fingers and then it happens to work so for compiling it's just a list of components and a key so if we think back to the example at the beginning where we had Advance State as the name of the transition method that we

have that's all we need it's that name Advanced State and so what this will I'll do in this line of code here is we'll do model. compile by key and so this is on your context model the whole model uh we give it a bunch of components W1 W1 IE and so on and so forth and then we tell it compile Advan State and so what this will do is it we'll go through and this just if we think to the the thing where we like composition of functions of everything together it will go

W1 goes through its Advanced State and then W1 IE goes through its Advanced State and then W1 eii goes through its Advanced State and it goes down the list now obviously if we reverse this whole list that will change the output it won't change what it's really doing but the order that the advances happen in does matter and it is dependent on the order that we find here

and so and the other Advantage is is that we can compile many of these so in this case we can see that we go through all two layers of our model here uh and

all of our neurons in our model
everything that's a z is a
neurons but let's just say we only
wanted to go through the first chunk
right we only wanted to do z_0 and then
we just wanted W_1 and then we just
wanted like z_1 that's all we wanted to
compute well we would have to do is we
would just have to put those three
values in a compile argument and it
would just go we can we can compile we
don't have to compile every component in
our model and what this means is that we
don't actually have to have every
transition defined for every component
for example when we're updating weight
matrices we tend to call it evolving
them right they're changing in time
they're
evolving we don't evolve neurons it's
not something that we do it's not
something that makes even doesn't make
sense in the context of most models so
from that standpoint we would only call
evolve on the
weights or we might only call reset on
the neurons if our weights don't
actually if our weights never have
anything static in them I mean variable
in them there's just like there's a
bunch of different things where we can
pick and choose which components and the
Order of the components that these
operations happen in and so in this
again just to fully explain we do have
two different output values coming out
of this compiled key this model.
compiled by key as well as one hidden

aspect that comes out of this model.

compiled by key so but first the AR the
outputs we have this Advance now Advance
here is the pure function returned by
the compiler uh so this is the function
that takes in the model State and all of
the arguments it's expecting and will
return to you the final model

State advv ARS these are all the
arguments it's expecting so as it goes
through and it compiles down W1 W1 iie
w1i and so on and so forth it's slowly
collecting a list of all of the keyword
arguments that it needs to actually
compute this transition so a W1 requires
T and z0 requires T and DT and Z2 e
requires a j what that means is that it
would end up you would have a list that
is we need T DT and J it if if two
things require the same parameter it
does assume that they're the same
parameter oh sorry the same yeah like
the same keyword argument um so if you
have t used across all of your
components you only have to pass in t
once so that's everything that's visibly
returned by this function in addition
what this function also does is we can
now call model. Advance State whatever
this compile key here or an optional
name flag that's not currently being
used we can take that advanced state and
we can do model. Advan State and it's
the same as calling model.

advance and this just has its benefits
so you don't have to necessarily keep
track of the actual advanc function if
you have a bunch of them you can just

name

them so that's everything on actually
how we get the compilation to run
so what if we want to perform many of
these State transitions in a row and
what I mean by that is if we think about
a spiking neural network we have a
forward pass where we take one look at
an image but also in spiking neural
networks usually we have to do many
forward passes up our image before we
actually can before we say we have an
output and so when we look at that
we're doing like 200 forward passes
before we ever actually care what the
output value is so because of that we
don't want to just do a state transition
from t_0 to T_1 get T_1 plug it back in
over here to get T_2 what I want to do is
I want to plug in t_0 here and then do T_1
 T_2 until I get to
 T_{200} and so for that we have scanners
now the NGC learn scanner is built on
top of Jax's scan function uh which is
effectively a method of doing a for Loop
uh inside of their compiled uh so inside
of their just in time
operations and the way this works is we
have our
scanner we have an observe function for
us but this is just whatever you want to
name it there's no actual hard fast
rules on what this needs to be called we
have compartment values or the model
State and then we have arcs
and so what the arguments are is these
are the keyword arguments for that
specific Loop in time so t_0 so at t_0 we

would have Δt is equal to zero and ΔT is equal to say one at T_2 or the second Loop through so yeah at T_1 or the second Loop through this R zero would be one and so on and so forth and we Define that down here in Loop ARs and we'll talk about that in a moment but then we'll notice that inside of this we're doing two actual transitions for every Loop we go through and we advance all of our state forward and then we do an evolve step over our model and this was done on an stdp or Spike timing dependent plasticity trained model um which means that we do an evolve Step at every single forward pass we do one for for every forward pass over an image we do an evolve step over the image and we'll notice that this gets a compartment values because this is a pure function that takes in the component the the model State and Returns the new model State evolve is another compiled method that takes in the current state and Returns the final State and then we just return the final State at the end of that Loop and it just loops and this will Loop over for however long you give it Arguments for so if we give it 20 200 Loops worth of arguments it will Loop 200 times and this part right here at the end is just how you can actually get an output out so this will return the compartment value located it S_r the RW Spike output of z_{1e} at every time step will be what underscore s is equal to

and finally uh to call it we just call
observe it's whatever the name of this
function is so if you named this Fu it
would just be model. Fu we tried to use
slightly more of that useful name so
here we have observe and here we have
observe then the loop arguments is a 2d
Matrix where every row is uh for every
row is just the list of arguments that
are found here so this is a 2 by 200
sorry a 200 by two Matrix
basically and that gets us our state
Transitions and so this is how we can
look at a model many many this is how we
can transition over many many many steps
with without ever having to come back to
Python and since this is
so scanners automatically put things
inside of just in time compilation just
for the way that they work internally
for JA but what this means is that we
can go from an incredibly slow forward
Passover one image to a compiled forward
Passover one image to a just in time
compiled pass over just like one pass
over an image to a just in time compiled
200 passes over an image and just do
that in a single function that we have
defined this is where all the speed
comes from
finally uh I just want to mention
monitors uh so what a monitor is is
sometimes we have a desire to actually
look at well the internal state of our
model as it's running and when we're
doing things with just in time
compilation we never get access to those
internal States because they all exist

in C and they never exist in Python and we can't look at them very easily uh again due to the way that just in time compilation works you can't put print statements in compiled code it does not work so what this means is that we have monitors and so the way this monitor works is it's just a component like any other component in NGC learn admittedly it does work differently under the hood but that's fine what we have is we have a monitor by name and then we have this default window length what this is doing is this will keep a rolling window of the last n values that you define so in this case 100 so it will keep the last 100 values of whatever you uh tell it to now this can now at some point you might say well why don't we just make this really long and do it for every value well that gets really big really fast if you're storing floating Point numbers and you're storing millions of them you can run out of memory in your computer um so you do have to be slightly careful with that but generally if you just keep the things that you're looking at to relatively small amounts you're be fine um and the use case for this we'll notice too is slightly different with the way we wire things into it so we'll notice that there isn't a compartment value right here this there's no m. compartment name we just wire directly into the Monitor and this is why I saying they work slightly differently um but as far as the outside

user is concerned you just wire into them like any other compartment you can think of it as one giant compartment that stores everything so here we take the spike output of z_{1e} and the spike output of z_{1i} and we wire them both into the monitor now somewhere in between this we have run like the observe function that has gone through and done a forward Passover and image now we want to actually look at the states as we're running and so what we'll notice here is in view the contents of it we can just do `m. View` and we just give it the compartment that we want to look at and this will give us the concatenated Matrix of all the spikes so if Z_{p1e} has 10 neurons in it it will produce 1 by 100 matrix like sets of neurons well basically just 10 by 100 matrix of spikes either zeros or ones and so as it goes through the output of the monitor will be 100 by 10 because that'll be all of the components of the spikes for all 100 time steps and so that gets us the ability to look at models as after after they have run but the internal states of the models as they're running um I will say there are a couple future things coming with monitors about the ability to actually get them to print out and integrate with map plot lib and getting it so you don't even actually ever have to view you can just make plots of compartments and they will just plot nicely for you um it is currently working mostly in our

development

Branch finally how has this been used right I've been marketing this to you as like hey this is the thing that we have made and it's really cool and it's really fast and it works really well but let's see it actually work in action so the paper that started this all was this paper called neuroc coding Frameworks for learning generative models by Dr aoria and kefir and so this was kind of the founding idea for NGC learn this was built in the original version of NGC learn about two years ago um and it built this incredibly complicated looking model here uh that has both excitatory and inhibitory connections as well as a bunch of lateral connections every line that we see here is part of the weight matrices and sometimes depending on the way that the connections go it's multiple sets of weight matrices um this is a model that you can build in NGC learn and it will look no more complicated than a bunch of little brackets telling comp compartments where to wire to in addition to that we've reproduced a deal and Cooks paper um from 2015 called unsupervised learning of Digit reconstruction using Spike timing dependent plasticity uh this is another model that we had this is more of a historic model that we have built uh inside mgc learn and gotten up and running without any problems at all we have uh one of my papers actually with my adviser Dr Aria called time

integrated Spike timing dependent
plasticity uh in which we built a
patching model uh that does what it that
basically is this whole structure where
we have a sensory input layer an image
um and it breaks up into a bunch of
different sub components uh subm models
basically that all learn small patches
of our image and then we have one final
part that Aggregates them all together
uh this sort of model is also very easy
to build an NGC learn and doesn't
there's no actual time difference
between running some of these um and so
again this is just something that we can
build and this is all able to be built
with components that already exist in
NGC learn if you wanted to build some of
these models this one the tddp synapse
isn't quite in there yet uh but this
model is fully in there you can build
this with every component that we have
inside of NGC learn um and it doesn't uh
without any without you having to write
anything yourself uh but even if you
wanted to rebuild this whole model
inside NG Seer and writing everything
yourself it still would not be that
challenging um in addition to that we
also have this contrasting signal
dependent plasticity which is a
self-supervised learning and Spike
neural circuits Again by my adviser uh
Dr Aria and this is again another model
that like this whole thing gets expanded
to this which does this and this model
is actually inside NGC learn as well all
of the components that are needed for it

are inside NGC learn and you can build it and you can look at it and it's all linked to in his paper and what that kind of wraps me up to here is the NGC learn Museum or just NGC Museum um and so NGC museum is a public repository for NGC learn that houses the biomic the brain Inspire Computing and the computational Neuroscience models basically everything that we've built uh for all of our models can be found here um it makes a very convenient place for if you have code that you want to publish somewhere um you can publish it here as just a hey here's your paper you can it's all it is organized and it's working on being more organized um into uh different groupings of stuff there's an whole exhibitor section for it so if you have multiple pieces of work that are all done using NGC learn you can you you'll have your own you'll have your ability to have your own folder that you can all that you can organize however you want that will have all of your papers that you've built together uh which would mean that if somebody is looking for similar papers that you have done they can come here and they can see ah here is a spike timing dependent plasticity paper that you've done and here's four other Spike timing dependent plasticity papers that you have also worked on as well as here's just other Spike timing dependent plasticity works by other authors uh that would you know allow you to cross reference things and just have uh a good uh Community built

platform for uh models because as I'm sure we're all aware it's really frustrating when you read a paper it has a really interesting method you go look to reproduce it yourself and the code is not available or it's in a git repo that's been deleted um I think we've all been there and so from that standpoint we're our goal is to make it so that that doesn't happen again and that everything built with NGC learn can be found all in one place so we can all see how much it can do uh and finally some upcoming features uh reinforcement learning uh so once again this is actually uh currently in our development Branch uh so we have grid-based environments uh technically you don't need to make them grid based it's just the easiest one to visualize uh so it's using components so with the way that components work uh if you can basically chunk your environment into its own individually Run State machine um you can display it and run it in NGC learn uh currently we have rat water maze rat te maze another model that I'm currently using for paper that's not publish yet uh that are all navigation based tasks that have to perceive the world around them and uh get updates I will say right now it is only limited to 2D uh you would not stop you from using it in 3D you just might have to see how to visualize it differently um and since it is built up out of components it is fully compilable uh and what that means is that we can compile the rat T maze

into a c function and integrate it directly with the compiled version of our um agent meaning that at the end of the day you can do your entire training over your RL environment in in C never having to touch itthon uh and I will say egocentric and World Views are both available so obviously these are rendering the world viiew and then the little highlighted section is your egocentric view both of them are very easily attainable um yeah and so with that uh there's some links to all everything and if anybody has any questions please don't hesitate to ask um slightly early on time but that was kind of the goal thank you William okay just while I'm getting back into the game here maybe just share a little bit of your journey to working on this like where are you at in grad school how how did you come on it yeah so I am a fourth year graduate student working under Dr Alexander Ria at the Rochester Institute of Technology um here in New York and so this kind of all started so my journey on spiking nlet started about two and a half years ago or so and very rapidly I found just like the speed problem to be incredibly annoying on like how do I actually get this to run in a way that's efficient um and as I started moving to bigger and bigger models specifically RL and some other models uh it started getting really really slow and so around that time so around like two years ago

or three years ago or so uh Dr abbia had started uh working on the first iteration of NGC learn which is what his original

paper this one was written using um and then I got a hold of it and it it works it works beautifully um but there since that Papo came out there were some pretty large advancements in the just computational things Jax had really taken off a little bit since that had come out um it was all originally built in

tensorflow uh which works but it really felt like we were hammering a square peg through a circular Hole uh to get it to do what we wanted and so starting about a year and a half ago now uh we kind of set out to make NGC learn what it is today and then probably eight months ago it really kind of took off on where it is with its compiling and the speed and we've been using it and developing it ever

since cool epic okay I'll start asking some questions from the live chat and if anybody wants to add more questions they can go for it

okay okay I'm going to read a possibly unrelated comment but if you have a thought otherwise it's not necessarily A a direct question okay Scott wrote lawyers say that contracts are a meeting of the minds singers and choruses experience aesthetic amplification of group identity here the meeting the meeting of voices in storytelling equals a smart mirror for group

identity that's

that's

okay it's interesting I do I do kind of

uh thinking about it is like just my

initial gut reaction to that is that uh

I think especially when we're doing

science sometimes we can get a little

bit too bogged down in being scientists

and not enough time in being like freeth

thinking and actually just coming up

with like new ideas right we get too

ingrained in like ruts of how things

have been done in the past and so I

think I think that's something that we

all do need to work towards more it's

just being a little bit more embracing

of just new ideas that might not

work awesome okay first question mobina

wrote does it only work with discreet

value

spiking no so I'm going to flip through

a couple of slides here so when you go

to these wires here there's nothing that

says that the wires actually move

discrete values but the advantage that

we run into is that the fact that they

can move discrete values so when we look

at here so the weight outputs so like w

like the weight weight outputs are

actual Matrix multiplications of weight

outputs we can see that here this is

this is not a discret value but for us

when we're dealing with spiking networks

the input

coming so the short answer to the

question is no it works fine with

floating Point values as well um it just

also works with discreet values which

many other systems do

not

okay to give you a breather I'm going to read an interesting quote that your last comment reminded me of it's from okay Henry David Thoreau in Walden he wrote I had not lived there a week before my feet wore a path from my door to the pond side and though it is 5 or six years since I trod it it is still quite distinct it is true I fear that others may have fallen into it and so helped to keep it open the surface of the Earth is soft and impressible by the feet of men and so with the paths which the Mind travels how worn and dusty then must be the highways of the world how deep the ruts of tradition and

Conformity it's a good quote it's a good quote I've seen his cabin I've been to I've been to that Lake cool like I had a sense that you would yeah yeah it's a it's a really cool place okay reading down viit your colleague and previous presenter wrote two questions so first question I saw that Jax also has a method. lower to lower the pure function down to machine code but not compiling do you see any fruitful exploitation of this function in the context of NGC simulation

Library um

possibly so that could get us definitely some of the beneficial parts so right now when you compile a function and then it throws an error debugging that error is a nightmare uh because it's all compiled under the hood I would have to

look more into it but if we can just get to machine code that would be really cool uh like if it didn't end up like if it ended up skipping over like it's going into C but I'm not sure if it does um on top of that something else that would be kind of interesting is and I've actually spoken to my father who also does computer programming about this is if we can get it all written into C or machine code theoretically you should be able to pre-compile this which means that you could get your entire model and all of the state transitions as a program basically that sits on your computer without actually ever having to touch python which I also think would be a really cool place to NV direction to go yeah I think I think the presentation and the package architecture reflects it like many octaves of computer science you're really getting down into the compilation details but understanding those is critical because the point you mentioned sort of by The Bu that there's the possibility with only some constraints to run a model on CPU GPU and neuromorphic that that that's an incredible cross um deployment of of potentially identical models so you could have certain models deployed across Hardware to better understand the hardware forget the model or of course for sure inside yeah inside of sim so inside of the tutorial section of like the

documentation for NGC learn there's a part with NGC lava and it actually will walk you through how to run it run the same model both on the the low he simulator and your computer like in the same time um and you can look and it will get the same output like it's got some Randomness to it so like your filter might be in a different spot but it will produce the same model yeah I mean what is it even like to or how is it similar and different to be programming on a neuromorphic simulator so the simulator is relatively straightforward actually going on to the physical Lei itself and this is why we have to stress Sim everywhere is a challenge so first off Intel's neurocore which is like their proprietary code for running on their Lei chips uh while it's proprietary code which means that I don't have easy access to it um like I have a couple working groups that I can have access to it through but it's not something that I can just put on my own machine and test um in addition to that I micro code is needed which is the basically what is usually used to like Pat CPUs and gpus and whatnot but the micro code it's the level between like machine code and it looks somewhat like assembly um you need to write a lot of your models in that which gets complicated so until we get a good translation layer between Python and either C and then C to that layer or between just python to that layer uh the

physically putting it on the hardware still is a hurdle yeah again super interesting how this is really spanning the gamut with different layers of computer science and and bringing in sure all these unconventional um compute okay another question from vit have you also considered automatic operator such as when we add two compartments A and B together instead of summation of a comma B it is a plus b and the library would wire them to the summation operation yes uh so yeah we definitely have considered that uh you run into so it works so so the long story short is like that can work the problem that you can run into is it can become a little bit syntactically uh weird to look at when you have compartment plus compartment meaning one thing and compartment. Value Plus compartment. value being another thing like basically the problem is that if we start overloading those operators it's you now have to be careful like right now if you try and add two compartments together uh and you forget to do like compartment. value um it'll just error because compartments don't have an addition with one another but if we produce an addition with one another um it starts to make it's you have a lot more silent errors though until we find a good way to overcome that problem it's currently on pause but this brings in a lot of interesting category Theory type

questions about concatenating nested systems and putting them in parallel in series and what are these operators what does it really mean when just to have a certain symbol and then what do you really want to do what are the operations supposed to do and that's kind of been why like we overrode the shifting Operator just because I wanted something there that was not just like do connect um but beyond that it's I've been a little bit hesitant to override them okay I'll read uh two comments from Professor orobia okay first first just my lab so far yeah but first first uh a historical not I guess the nod from will to the old many years ago NGC learn was nice wink the original NGC learn was written in tensorflow and started by offering generic customizable support for predictive coding first note and then second note um I posted a link to where we've curated uh many implementations of active inference and Alex we would love to have Community contributions translating these active inference models to NGC learn we would love to feature and maintain these in the NGC Museum I mean that was an incredible insight s to have a package Centric repository yeah yeah the the whole goal of this so all four of these models that are shown here are all in the museum already or on their way to being in the museum and yeah the goal is just so that way like if you guys have this repository if here's all the papers well

what's better than here's all the papers
then here's all the papers and the code
to reproduce them all in one place
yeah um okay you you mentioned a few
pieces of this but uh mobina wrote what
improvements are currently being
explored to enhance NGC
learning yeah so uh again one of the
main ones is just getting reinforcement
learning up there and available for
people to use easily um in addition to
that there's an overhaul of our visualiz
a tool tools that I've want to have so
we already support um things like the
raster plots uh all of like for like
plotting Spike patterns uh we support
obviously Matt plot lib does all of its
stuff and since OB all of our NC learn
stuff is built inside of numpy it all
plugs and plays into that really easily
um but we do have um a couple other just
additions to all of this that we're
trying to get up and running basically
there's an overhaul to the visualization
Library
um in addition to that I'm sure there's
other projects that I've been thinking
about but I don't have my I don't have
my notebook with my list of plans in
front of me but those two are probably
the first two on my list is the
reinforcement learning and then
visualizations cool and this is a
question and the help section and the
library built help section yes the lobby
of the museum um yeah and then the art
outside um here's a question that we've
discussed with each of the previous

package developers today uh active inf.
JL and and the RX and fur like what was
the most challenging part of developing
this
out
um definitely the
compiler and more and it's not the funny
thing is it's not the compiler
specifically that was the most
challenging part of developing this out
it was how did I want the user to
actually interface with it because one
of the most complicated things and I can
go back
to here right was getting this to all
work in a way that didn't feel
clunky so and it still isn't perfect
it's just the best I've we've figured
out so far but the the way to define
your compartment in your fixed
parameters and then write transition
functions that can pull them into into
into itself and do the operation getting
that workflow down in a way that made it
so that way if you want to build a new
component it doesn't take very long was
a challenge um that was probably the
most challenging part
um the next one's definitely just
actually getting the simulator to plug
andplay everything together um I could
give another like 3our talk on how the
simulator like the NGC simli actually
works under the hood um because
it's it does a lot of metaprogramming
and pattern matching and function
generation like your actual compiled
method that you're running is like four

step three steps of of like dynamically
created functions deep basically and so
getting that to all work without bugs
was hard um and it's been doing pretty
well uh it's all we've only had two very
very minor bugs in the past like five
months so I'll call that a success in
analogy we love minor
bugs oh yeah of course um okay I'll read
another question roric wrote is there an
active outreach program to the museum as
such treasure ought not to be buried
so we're not actively going out and
trying to seek things but it is open
source and anybody can make poll
requests into it um so and we are and
whatever we for like our lab or like for
our own personal research reproduce a
model um for either a baseline or just
as like as we're learning something new
that also ends up in there but um I
think as the going out to people
specifically and being like Hey we're
going to it's a lot of it is as we're
we're trying to get the people that use
our tool to just when they've published
paper there that's a very humble answer
also you're doing the active part now so
that can't
be okay I'm going to read another this
is not a question from Scott but just if
you have any thoughts okay he just wrote
things get interesting as our new
computational and information processing
systems computers internet AI etc etc
etc more Faithfully reflect human
cognition active inference and
biophysical models than are existing

institutions and organizations the latter are historical artifacts not yet improved and socialized to new learnings about physics supported flows I mean I think I think this is just a my first thought of that is uh many yeah many many modern AI systems and giant structures that are made do not reflect well on how on like are not faithful to biology um and we definitely have a we should definitely have a goal on making them more faithful to biology and uh getting a basically there's a bunch of things that like biological models have uh can achieve for us uh with one of the biggest ones just being the Energy Efficiency of neuromorphic computing and the fact that we have things like chat GPT and whatnot that have uh you know use power draws of small cities to train uh kind of goes against all of that and so I do think that like this is definitely a way of the future I don't think it will ever beat out everything but I do think it's definitely something that we do need to continue pursuit of okay I I have a question what is the cutive or possible title or topic of your dissertation overall like is this one puzzle piece or is this the so so yeah so I've done my dissertation proposal I off the top of my head I do not remember what my proposal name was um but so I do uh spiking representation learning uh so I up until now most of my focus has been specifically on the uh

learning rules in credit assignment
plasticity rules of spiking neet
specifically Spike timing dependent
plasticity uh this is my paper um time
time integrated Spike timing dependent
plasticity um and then uh I wanted to do
all of this with the direct goal of
putting it into reinforcement learning
uh which is why one of the reinforcement
learning parts of NGC learn is on the
way is because I need it for my research
and since other people have also
expressed an interest in it for their
research I'm like let's all put it in
the library together um and and so yeah
so I've got that and then the actual uh
reinforce and then all of this basically
played together to go with the
representation learning of how do humans
actually represent stuff because I'm not
a fan of directly going from like pixels
to thought process I want to go pixels
to representation to thought process and
so that's kind of where I've been at
right now in my career
path cool okay another comment from Alex
you wrote
we should also mention that njc learn is
going to offer support for dynamical
systems and even more detailed support
for differential equations beyond the
basics we have now smiley face yes yes
that is that is the thing too that we
are on on track for we've got um
currently a bunch of different
integration methods that are in there um
but they don't allow they allow for some
complex dynamical systems but not every

complex dynamical system that we want
and so the goal is to be able to put all
of those in there too um pretty much
anything you think of that can be
represented as a state machine is
something that we would love to be able
to support
um machine moving pieces around and so
while it works really well for spiking
neural Nets it works really well for so
many other things that use uh that are
you know a dynamical system something
that changes with time yeah from the
active inference side when I see that
well first the reinforcement learning
and the ability to have the egocentric
and the allocentric I think is very
fascinating and and it makes me wonder
how how do
epistemic
value inferences come into the
picture when we zoom out Beyond utility
or reward maximization and include these
networks with structural fixity or with
with a dynamic structure you're doing
learning that's kind of just an
interesting yeah for sure for sure I
don't I don't have any great Insight on
that right now other than it sounds
really cool and I would love to be there
working on it cool well I I I don't know
if I should say that I do or don't hope
your PhD ends before
then do you have any uh I hope to get oh
work on
this I'm pretty sure even after my phdns
I'm still going to be working on this so
I it's all good cool do you have any uh

last comments or thoughts to
add uh not really just that if you are
ever trying to set up NGC learn and it's
you've you're struggling with anything
don't do not hesitate to reach out um to
either uh myself or my adviser Dr Oria
both of us are the on the main Dev team
of NGC learn and we would be more than
happy to help okay I'll just read a
closing comment from Dr Robia we really
hope the active inference community and
audience considers Community
contributions to NGC learn especially if
they want to go beyond using backprop
slst standard deeplearning also consider
using it in the classroom to teach
computational
Neuroscience cool all right excellent
thank you William farewell